Refactoring tools

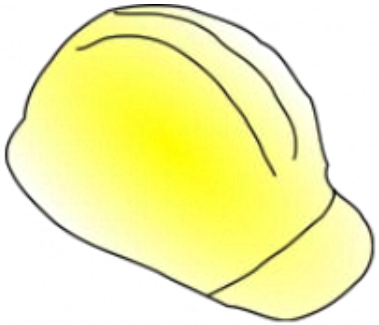
Alejandra Garrido
Objetos 2
Facultad de Informática - UNLP

Cuando aplicar refactoring

- Cuando se descubre código con mal olor, aprovechando la oportunidad
  - dejarlo al menos un poco mejor, dependiendo del tiempo que lleve y de lo que esté haciendo
- Cuando no puedo entender el código
  - aprovechar el momento en que lo logro entender
- Cuando encuentro una mejor manera de codificar algo

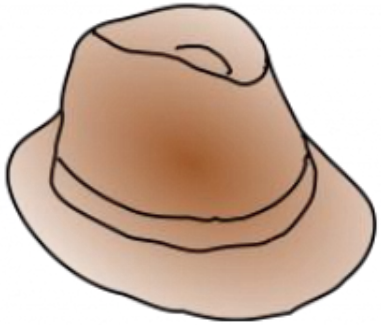
The 2 hats
La métafora de los 2 sombreros

Kent Beck



Adding Function

Se exploran ideas, se corrigen bugs



Refactoring

Solo puedo refactorizar con tests en verde

Puedo cambiar de sombrero frecuentemente
Pero solo puedo usar **1 sombrero por vez**

Automatización del refactoring

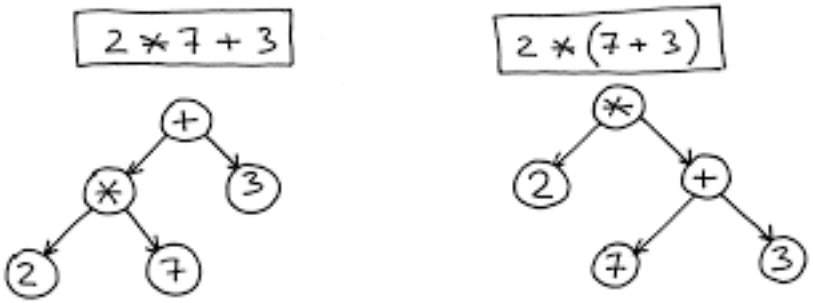
- Refactorizar a mano es demasiado costoso: lleva tiempo y puede introducir errores
- Herramientas de refactoring automático:
  - chequean precondiciones
  - realizan transformacione
- Características de las herramientas:
  - potentes para realizar refactorings útiles
  - restrictivas para preservar comportamiento del programa (precondiciones)
  - interactivas, de manera que el chequeo de precondiciones no debe ser extenso

Copyright Alejandra Garrido - LIFIA - UNLP

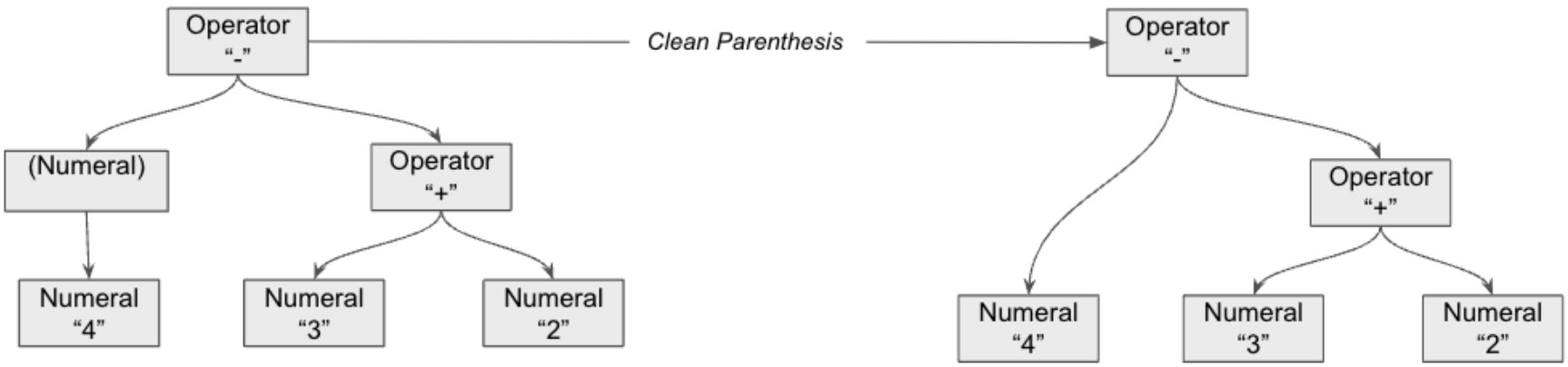
La clave de la automatización: el Abstract Syntax Tree (AST)

- Las precondiciones se chequean sobre el AST + symbol table
  - por ej. subárbol completo es unidad que se puede extraer
  - alcance de variables se chequea en tabla de símbolos

  ⇒ lo que no se puede analizar en estas estructuras **no se chequea**!!! Unit testing es fundamental
- Las transformaciones se realizan mediante AST rewriting



Ejemplo de AST rewriting



Pretty print ((4) - (3 + 2))

Pretty print (4 - (3 + 2))

Progreso de las herramientas

- 1ra herramienta de refactoring automático: Refactoring Browser (RB) para Smalltalk
  - John Brant & Don Roberts, doctorandos de Ralph Johnson.
  - 1er adoptante: Kent Beck y su empresa
- Luego: Code Critic para Smalltalk: 1ra herramienta de detección de code smells
- Luego todas las IDEs adoptaron la misma estrategia de transformación que RB
- Hoy en día: IDEs+IA que sugieren donde aplicar refactorings, pero con alto grado de alucinaciones

¿Por qué refactoring es importante?

- Ganar en la comprensión del código
- Reducir el costo de mantenimiento debido a los cambios inevitables que sufrirá el sistema
  (por ejemplo, código duplicado que haya que cambiar)
- Facilitar la detección de bugs
- La clave: poder agregar funcionalidad más rápido después de refactorizar

Referencias y videos

- “Big Ball of Mud”. Brian Foote and Joe Yoder. Pattern Languages of Programs 4. Addison-Wesley 2000.
- Big ball of mud @ Google Talks 2007: https://www.youtube.com/watch?v=LH_e8NfNV-c&t=75s
- “Refactoring”. Martin Fowler. Addison-Wesley 1999. (original con ejemplos en Java)
- Martin Fowler @ OOP2014 "Workflows of Refactoring“: https://www.youtube.com/watch?v=vqEg37e4Mkw

Más referencias y videos

- Catálogo de refactoring: https://refactoring.com
- Categorías de code smells: https://sourcemaking.com/refactoring/smells
- Code Refactoring: Learn Code Smells And Level Up Your Game!: https://www.youtube.com/watch?v=MM6_tyvBRXE
- “Object Oriented Metrics in Practice”. Lanza & Marinescu. Springer 2006